

دليل ال Structure الجديد — إزاي تستخدمه

من ٨ commands ضخمة (٢٤١١ سطر) ل structure موزّع نظيف (٥٦٠ سطر) بنفس الوظائف وأكثر

إيه اللي اتعمل بالضبط

ال zip فيه .claude/ كاملة + CLAUDE.md محدّث. الوظائف كلها موجودة، بس اتوزّعت على الطبقات الصح: القواعد الثابتة في .rules/، ال workflows في .skills/، المهام الثقيلة في .agents/، والإلزام في .hooks/. التكرار اتشال بالكامل – ال Rails rules بقت في مكان واحد بدل ٣.

Structure ال

```
.claude/
├── rules/                ← أوتوماتيك حسب الملف اللي بتلمسه
│   ├── rails-conventions.md    paths: app/**, lib/**
│   ├── rspec.md               paths: spec/**
│   └── api-endpoints.md        paths: routes.rb, controllers/api/**, serializers/**
├── skills/              ← workflows (بدل commands/)
│   ├── plan/SKILL.md         (plan-implementation كان)
│   ├── implement/SKILL.md     (implement-plan + implement-plan-nicely كان)
│   │   └── references/        architecture-discussion.md · dashboard.md
│   ├── fix-bug/SKILL.md
│   ├── review/SKILL.md
│   │   └── references/        checklist.md · html-report.md
│   ├── document/SKILL.md      (documentation كان)
│   │   └── references/        confluence-page.md · rswag.md
│   ├── analyze-requirements/SKILL.md
│   └── full-cycle/SKILL.md     (orchestrator بال subagents)
├── agents/              ← منفصل context بـ subagents
│   ├── requirements-analyst.md · code-reviewer.md · doc-writer.md
├── hooks/               ← إلزام حتمي
│   ├── block-debug-statements.sh · api-change-reminder.sh
└── settings.json        ← hooks تسجيل ال
CLAUDE.md               ← workflow بتاعك + ٤ سطور عن ال
```

إزاي كل طبقة بتشتغل

rules – لو Claude فتح أو عدّل ملف بيطابق الـ `paths` ، القاعدة بتتحمل لوحدها. يعني وانت بتكتب `service object`، الـ `Rails rules` موجودة في دماغه من غير ما تقول حاجة. ولما يكتب `spec`، الـ `RSpec rules` بتظهر.

skills – الـ `description` هو الـ `trigger`. بتشتغل بـ `/name` أو لوحدها لما كلامك يطابق الـ `description` (إلا لو `disable-model-invocation: true`). الـ `references/` بتتقرا بس لما الـ `skill` تشتغل → بتوفر `context`.

agents – لما `skill` تقول "Claude، delegate to X"، الـ `context` جديد نظيف، الـ `agent` يشتغل ويرجع ملخص قصير بس. الـ `Jira/Figma dumps` مبتلّوش الـ `session` بتاعتك.

hooks – سكريبت `shell` بيتشغل قبل/بعد أداة معينة. `exit 2` = يمنع (في `PreToolUse`) أو يبلّغ Claude (في `PostToolUse`). حتمي ١٠٠٪ مش معتمد على إن Claude يفتكر.

٢ التركيب — ٦ خطوات

- Backup** – `git stash` أو `branch` جديد. الـ `zip` بيستبدل `CLAUDE.md` – قارنه بنسختك (أضفت ٤ سطور بس تحت "Backend workflow").
- فكّ الـ zip في root المشروع** – بيعمل `./claude/`. جنب `AGENTS.md` بتاعكم. لو عندكم `./claude/rules/`. أصلاً، تأكد الأسامي متضاربش.
- امسح الـ commands القديمة** – `./claude/commands/*.md`. لـ ٨ ملفات اللي اتحوّلت. لو سيبتهم، الـ `skill` بنفس الاسم بتأخذ الأولوية بس `plan-implementation` و `implement-plan` هيفضلوا ظاهرين وبلخبطوا.
- chmod +x** لـ `hooks` : `chmod +x ./claude/hooks/*.sh` . الـ `zip` حافظ الصلاحيات بس تأكد.
- Restart Claude Code** وشغل `/skills` – لازم تشوف الـ `skills` و `/hooks` – لازم تشوف الـ `agents` – لازم تشوف الـ ٣.
- اختبر hook واحد** – حط `binding.pry` في ملف، `git add`، وقول لـ "Claude" `commit`. لازم يتبلوك برسالة واضحة.

٣ حاجات لازم تتأكد منها ضد الـ docs الرسمية

- صيغة الـ `paths:` في الـ `rules` – استخدمت `YAML list`. بُص على أي `rule` موجود عندكم أصلاً وطابق صيغته لو مختلفة.
- صيغة الـ `allowed-tools` – استخدمت `Bash(git diff:*)` pattern. لو Claude لسه بيسألك `permission`، شيل السطر خالص وسيب الـ `default`.
- الـ `context: fork` – استخدمته **مش** في الـ `frontmatter` (فيه `bug` مبلّغ عنه إنه أحياناً بيتجاهل). بدلها الـ `full-cycle` بيقول صريح "delegate to agent X" وده أضمن. لو الـ `bug` اتصلّح عندك تقدر تضيفه.

تلقائي / plan/

```
/plan https://planradar.atlassian.net/browse/PROJ-123 --confluence <url> --thoughts "background job with retries"
/plan .plans/implementation-plan-PROJ-123-2026-09-13.md --thoughts "bulk delete locks table, use batched soft deletes"
```

Full mode بـ Jira URLs، أو Revision mode بـ path لخطة موجودة. الخطة بتتخفظ في `.plans/`. ال Revision عمره ما بيكتب فوق الأصل – بيعمل `-rev-N`. وتقدر تقول عادي "plan this story PROJ-123" وهيشتغل لوحده.

يدوي فقط / implement/

```
/implement (implementation-plan-* أو bugfix-*) .plans/ بياخد أحدث خطة في ←
/implement .plans/xxx.md --dashboard
```

يدوي عن قصد لأنه بيكتب كود. بيناقش ال architecture (٦ مواضيع واحد واحد بـ `AskUserQuestion`) وبعدين phase بـ phase gate بينهم. ال `--dashboard` بيعمل HTML واحد بيتجدد كل خطوة – إنت تعمل refresh، مفيش polling.

تلقائي / fix-bug/

```
/fix-bug https://planradar.atlassian.net/browse/PROJ-456 --thoughts "fails only with 50+ projects"
```

بيحقق في ال root cause، يعرضه عليك، ولما توافق بيكتب `.plans/bugfix-PROJ-456-<date>.md`. بعدها `/implement` هياخده لوحده.

تلقائي / review/

```
/review ← local uncommitted changes
/review --plan .plans/xxx.md ← + عرّض revision لو ال fixes
/review https://gitlab.com/planradar/x/-/merge_requests/123 ← remote, comment only
```

لاحظ في أول ال SKILL.md: `!git status --short` – ده **dynamic injection**: بيتنفذ قبل ما Claude يقرأ ال prompt فبيشوف حالة ال repo فورًا. ال checklist العشرة في `references/`. Remote mode مبيعّدلش كود أبدًا.

document/ يدوي فقط

```
/document --new https://planradar.atlassian.net/wiki/spaces/PMT/folder/4040359987 .plans/xxx.md
/document https://planradar.atlassian.net/wiki/spaces/PMT/pages/4031676438/... .plans/xxx.md --thoughts "..."
```

Swagger/rswag الأول دائماً وبعدين Update mode Confluence. يعمل diff-review HTML وإنت تختار الأرقام اللي تتطبق.

analyze-requirements/ يدوي فقط

```
/analyze-requirements PROJ-123 PROJ-456 --figma <url> --check-linked-tasks
```

يدوي لأنه ينشر على Confluence. نفس المنطق القديم بالظبط بس أنحف.

full-cycle/ يدوي فقط

```
/full-cycle PROJ-123 PROJ-456 --figma <url>
```

التغيير الأكبر. المراحل الثقيلة في القراءة (requirements, review, docs) بتروح لـ **subagents** وبترجع ملخص بس. المراحل التفاعلية (plan, implement) بتشتغل معاك مباشرة. وفيه **state file** `.plans/full-cycle-<ts>.md` هو المصدر الوحيد للقرارات – Claude بيرجع يقراه بدل ما يعتمد على ذاكرته. ده اللي بيحل مشكلة "نسي قرار المرحلة الأولى".

Hooks ال ٤

Hook	بيتشغل إمتى	بيعمل إيه
block-debug-statements.sh	PreToolUse على Bash، لو الأمر فيه <code>git commit</code>	يفحص ال staged Ruby diff على <code>binding.pry / byebug</code> و <code>puts / debugger</code> لو لقي <code>exit 2</code> و <code>commit</code> مبيحصلش. اتاختر وشغال.
api-change-reminder.sh	PostToolUse على Edit/Write لملف <code>routes.rb</code> أو <code>controllers/api/</code>	بيعت ل Claude تذكير صريح بال permissions و rswag و V2. مش بيمنع، بس Claude بيشفوفه فوراً.

اللاتين شغالين من غير `jq` (فيه fallback بـ `sed`). لو عايز تضيف hook تاني: سكريبت في `hooks/` + entry في `settings.json`.

٥ إيه اللي اتشال أو اتغير عن قصد

- **Hotwire/Tailwind rules** – كانت منسوخة من template عام وملهاش علاقة بـ API backend. شلتها. لو عندكم views فعلاً، ضيفها في `rails-conventions.md`.
- **Live polling dashboard** – استبدل بـ HTML واحد بيتجدد (الأصل مكانش هيشغل من `file://`).
- **Parsing rules الطويلة** – اتشالت من كل skill. `argument-hint` + وصف قصير كفاية، Claude بي فهم الـ flags.
- **الأسامي** – `plan / implement / document` بدل الأسامي الطويلة المتشابهة. لو الـ skills عندكم في plugin namespace (زي `backend:` الي ظهر في fix-bug القديم)، الاستدعاء يبقى `/backend:plan`.
- **الملفات كلها بتتحفظ في** `plans/` – `dashboards` و `reviews` و `reports`، مش في root المشروع. أنصف.

القاعدة اللي تمشي عليها من هنا ورايح

قبل ما تضيف أي حاجة اسأل: **لازم تتنفذ دايمًا؟** → `hook`. **قاعدة ثابتة عن نوع ملف؟** → `rule` بـ `paths`. **workflow** **بخطوات؟** → `skill`. **مهمة ثقيلة في القراءة ومحتاجة ملخص بس؟** → `agent`. **حاجة عن المشروع كله وقصيرة؟** → `CLAUDE.md`. راجع شهريًا وامسح اللي مبتستخدمه.

الملفات مبنية على (`skills`, `rules`, `subagents`, `hooks`) docs.claude.com بتاريخ سبتمبر ٢٠٢٦. النقاط الثلاثة في الصندوق الأصفر لازم تتأكد منها في بيتك قبل الاعتماد الكامل.